AgenticBot: Conversation Engine Plan

Voice bot for Chughtai Lab's call center. Draft for review and handover.

Summary

- **Goal:** an Urdu and Roman Urdu voice bot for the call center. It runs workflows for doctors, home sampling, tests, home care and center services, lets the caller switch between them mid-call, and answers questions about centers, doctors and tests. Anything unsafe goes to a human.
- **Data first:** the source data is unstructured. It is extracted into approved tables (centers, doctors, tests, home care, center services, cards, promotions) before anything else depends on it.
- **Architecture:** one Python service on LiveKit. Asterisk trunks SIP into LiveKit, and the functions of the existing Rust implementation are rebuilt in Python. The hosted Urdu STT and TTS and Qwen3-8B are called by the bot. Two checks (4.2) confirm nothing needed is lost.
- **Engine:** our own workflow engine (4.4). The LLM only extracts names and helps routing; code owns the flow and every side effect; replies are templated Roman Urdu.
- **Knowledge:** facts (fees, timings, charges) come from Postgres tables. RAG over approved documents in `pgvector` covers free text. Conversation state is in Postgres.
- **Switching:** a router runs every turn. A switch suspends the current workflow with its slots and offers to resume afterwards (8.1).
- **New workflows:** a new workflow is a folder with a manifest; the router reads the registry, so it needs no code change (14).
- **Safety:** nothing is submitted without explicit confirmation, high-risk slots are read back, and medical advice and complaints go to a human.
- **Phases:** (1) data structuring, (2) LiveKit and SIP with checks C1 and C2, (3) real endpoints, (4) workflow foundation, (5) real STT tuning, (6) remaining workflows, (7) facts and RAG, (8) safety and call control, (9) load test, (10) shadow pilot.
- **Open:** the raw data and its approver, Asterisk with LiveKit SIP, what else the Rust code does, the TTS input script, booking APIs, and where human transfers go (section 16).

0. Requirements from the Chughtai analysis

The source data is highly unstructured and must be structured before it is ingested into the knowledge base. The bot pulls doctor details, fees, timings, test charges and promotions from this data during calls, so data structuring is the first piece of work (section 13).

0.1 Data to structure

Entity	Fields (as specified)
Centers	names, location, city, address, contact, cards, promotions, discounts
Doctors	names, qualification, fee, timing, days
Tests	names, protocols, charges, discounts, doctors, `IsHomeSamplingAvailable`

Additions the workflows need (to confirm): - **Doctor to center link**, to answer "Dr X kahan baithte hain?". - **Home care services** (nurse visit, injections, drips: name, charges, availability, areas) and **center services** (radiology, cards: name, centers offering it, charges, booking rules). - **Validity dates and a source reference** on promotions and discounts, so an expired offer is never quoted. - **Relations:** test to doctors, doctor to center, discounts and promotions to tests, centers and cards.

0.2 Facts during calls

The bot resolves the name the caller says against the structured data and reads the stored value ("Dr X ki fee kitni hai?", "CBC ka charge kya hai?"). Fees, timings, days, charges, discounts and addresses are never generated by the LLM. If a field is missing, the bot says so and offers a human.

0.3 Workflow categories

As specified: design workflows around five categories, each with its own sub-workflows.

1. Doctors workflows (consultancy)
2. Home sampling workflows (a generic one for all cases)
3. Test workflows (test rates, protocols, reschedule, cancel, results inquiry, discounts)
4. Home care workflows (nurse, injections, drips, etc.)
5. Center services workflows (radiologist, cards, etc.)

Sub-workflows beyond what was specified are proposals.

#	Category	Sub-workflows	Source
1	Doctors	Doctor details (qualification, fee, timing, days); find a doctor; book, reschedule or cancel a consultation	Category given; sub-workflows proposed
2	Home sampling (generic)	Book a request; fasting and preparation guidance; coverage check	Category given; sub-workflows proposed
3	Test workflows	Rates; protocols; reschedule; cancel; results inquiry; discounts	Given
4	Home care	Nurse visit; injections; drips; others as added	Categories given; list to extend
5	Center services	Radiology; cards; others as added	Categories given; list to extend

Cross-cutting: FAQ answers, human handoff, and switching between any workflow mid-call.

0.4 Workflow modules to build

Each module is a workflow folder (section 8). Status is against the prototype in `voicebot/`.

Module	Category	Covers	Reads (7.3)	Writes via API	Status
W1 `doctor_info`	1	Qualification, fee, timing, days, where a doctor sits; find a doctor	Doctors, Centers	None	New
W2 `doctor_booking`	1	Book, reschedule, cancel a consultation	Doctors, Centers	Consultation booking	New
W3 `home_sampling`	2	Generic booking, coverage check, fasting and preparation guidance	Tests, areas, Centers, discounts	Home sampling request	In prototype; needs real data
W4 `test_info`	3	Rates, protocols, discounts	Tests, discounts, Doctors	None	Partial (`lookup` answers price); needs protocols, discounts
W5 `booking_changes`	3	Reschedule and cancel	Booking API	Reschedule, cancel	Partial (`booking_status`); placement across categories open

Module	Category	Covers	Reads (7.3)	Writes via API	Status
W6 `results_inquiry`	3	Results status	Booking and results API	None	New; needs approved verification rule
W7 `home_care`	4	Nurse, injections, drips, others	Home care services, areas	Home care request	New
W8 `center_services`	5	Radiology, cards, others	Center services, Centers, cards	Appointment or request, where it exists	New

Shared modules: entity resolution `resolve_entity` (in prototype); router and session manager (in prototype; needs real-utterance tuning); booking pattern of slots, read-back, confirmation and one API call (in prototype); verification by booking reference plus phone (partial, in `booking_status`); FAQ and lookup answers (in prototype; needs real content); human handoff (in flow; real transfer needs call control, T4). Sub-workflow lists for W1, W2, W3, W7 and W8 are proposals; confirm with the business team before building.

1. Purpose and scope

AgenticBot answers inbound calls in Urdu and Roman Urdu. It runs workflows across the five categories, lets the caller switch between them mid-call, and answers questions about centers, doctors, tests, home care and center services from a structured knowledge base. Anything it cannot handle safely goes to a human.

In scope: the whole call path in one Python service on LiveKit: SIP call handling, caller number and caller-details lookup, the voice pipeline (VAD, calls to the STT and TTS endpoints), the conversation engine (understanding, flow control, knowledge lookup, replies, handoff decisions), and call control (transfer, hangup, SMS).

Out of scope: the Asterisk PBX itself (assumed to stay and trunk calls into LiveKit; to confirm); the STT, TTS and LLM models, hosted separately (the STT and TTS are the endpoints the prototype uses); the external business APIs.

Design stance: a hybrid, not a free-form agent. Code owns the flow and every side effect. The LLM only helps understand noisy text.

2. What exists today

2.1 Existing Rust implementation and where each piece goes

Rust today	In the LiveKit design
Inbound calls via Asterisk	Asterisk trunks SIP into the LiveKit SIP service; an inbound trunk and dispatch rule create one room per caller (T1)
Caller number	Read from the SIP participant (attribute to verify) (T2)
Caller details from external APIs	Same API calls ported to a Python client, run at call start (T3)
Incoming audio to the Urdu STT	The caller's audio track goes to the agent, which sends utterances to the STT endpoint (V3)
Output connection for the TTS	The agent publishes the TTS audio into the room (V4)
Anything else	Unknown until check C1 (4.2)

2.2 Prototype in `voicebot/` (Python)

Built and run 2026-10-22 against text and fake STT/TTS endpoints. See PROGRESS.md. It has two workflows (`home_sampling`, `booking_status`) with switching, suspend and resume; a router (rules, then Qwen3-8B); `datakit` (CSV loader, alias, fuzzy and phonetic matching, spoken "which one?" step); a FAQ layer with 33 sample entries; a mock booking backend; a test console UI; a LiveKit voice loop exercised against fake STT/TTS (booking, switch and resume, barge-in scenarios); and test assets (scenario files, unit tests, a noise evaluation script).

Not yet exercised: the real STT, TTS and LLM endpoints, real-call accuracy, a real phone line, SIP, and simultaneous calls. Prototype data (tests, areas, prices, FAQ) is placeholder.

Coverage against 0.3: `home_sampling` is category 2; `booking_status` is the reschedule and cancel part of category 3. Not built: doctors, home care, center services, test rates and protocols, discounts, results inquiry. The prototype data has only tests, areas, branches and pricing.

3. Design inputs from the prototype

1. The conversation core is separate from audio: `app/bot.py`, `flow/`, `router/`, `datakit/`, `faq.py` and `backend/` run without audio or LiveKit. Only `agent.py` and `services/` touch LiveKit and the STT/TTS endpoints.
2. Replies are templated Roman Urdu. The LLM does name extraction and routing only.
3. A workflow is a `manifest.yaml`, a `workflow.yaml` and an optional `hooks.py`.
4. The router uses rules first, then Qwen3-8B. A Laya router adapter exists but is not used by default.
5. STT: use the HTTP `chughtai` endpoint with per-utterance chunks.
6. STT output will contain errors on names, addresses, phone digits and test names. Every error must be recoverable: read-back, closed lists, retry limits, human handoff.
7. Scenario YAML files are the regression suite; every prompt, threshold or data change is re-checked against them.

4. Architecture

One service, one framework. The bot is a Python service on LiveKit that owns the whole call, from the SIP call arriving to the reply audio going back. STT, TTS and the LLM are hosted separately and called by the bot. It can be tested end to end with a simulated caller.

```
Caller -> phone network -> Asterisk PBX -> SIP trunk
   -> [LiveKit SIP service] -> [LiveKit server: one room per call]
   -> [Agent worker, Python]
        At call start: caller number from the SIP participant;
                       caller details from the external APIs; session created
        1. VAD / turn detection cuts the caller's speech into utterances
        2. STT endpoint (hosted separately)  -> Roman Urdu text
        3. Normalizer + conversation engine
             Session store -> Router -> Workflow | FAQ | Lookup | RAG | Human
             Business APIs, Knowledge base
             LLM endpoint (Qwen3-8B, hosted separately): name extraction + routing only
        4. Templated Roman Urdu reply (converted to Urdu script if the TTS needs it)
        5. TTS endpoint (hosted separately)  -> audio published into the room
        6. Call control: transfer to a human (SIP REFER), hang up, send SMS
```

4.1 Roles of STT, TTS and the LLM

Piece	Role
STT (hosted separately; the prototype's endpoint)	Urdu model that outputs **Roman Urdu**, so the engine works in Roman Urdu end to end. HTTP `chughtai` endpoint, one request per utterance.
TTS (hosted separately; the prototype's endpoint)	Urdu model. Input script (Roman Urdu or Urdu script) to be confirmed with a test; if Urdu script, a Roman-to-Urdu conversion step is needed (E11).
LLM (Qwen3-8B on vLLM, OpenAI-compatible, thinking off)	Extracts names from noisy text and decides routing when rules are unsure. It never writes replies, picks the next step, calls an API or states a fact (one proposed exception: compressing retrieved text, 7.4).
Engine code	State machines, slot validation, lookups, confirmation, submit, handoff, templated replies.

4.2 Framework decision and checks

Decision: LiveKit Agents, self-hosted. Basis: - LiveKit documents SIP inbound trunks, dispatch rules that give each caller their own room, and cold transfer by SIP REFER. - The prototype has the LiveKit voice loop (VAD, STT and TTS plugins, barge-in) and `deploy/` compose files. - No separate audio link between a Rust gateway and the bot is needed. - One language and one codebase, with the conversation engine unchanged.

Pipecat (WebSocket transport) was considered; it is the contingency if the SIP path fails. **Cost:** a LiveKit server, the SIP service and Redis to run; a host with a public IP; UDP 5060 and 10000 to 20000 open; rebuilding the functions of the existing Rust implementation that are still needed.

Two checks:

Check	Work	Pass condition
C1. Rust inventory	With the Rust team, list everything the Rust service does beyond the five items in 2.1 (for example agent-desk transfer, queueing, DTMF, recording, monitoring, billing). Mark each needed (rebuild in Python) or dropped.	Every needed function has a home in section 6
C2. SIP spike	Trunk Asterisk into a self-hosted LiveKit SIP service on a dev host with a public IP; the prototype agent answers a real call.	Call connects; caller number arrives; 8 kHz audio quality acceptable; REFER transfer to the agent desk works with your Asterisk

If C2 fails: keep the existing Rust layer as the audio gateway, with a Pipecat voice service over a WebSocket audio link. The engine is framework-free, so only the voice glue changes (`agent.py` , `services/`). Links and examples: section 17. Check the current docs of the release you pin.

Steps to take the prototype to real calls:

Step	Work	Gate
S1	Stand up LiveKit server, SIP service and Redis on a dev host from `deploy/`	Agent registers; a browser test call works
S2	SIP trunk from Asterisk to LiveKit; inbound trunk and dispatch rule (C2)	A real call reaches the agent; caller number arrives
S3	Port the Rust caller-details API calls to a Python client at call start; map to `Bot.new_session`	Greeting uses the right session; known details not asked again
S4	Real STT and TTS (tokens needed); `scripts/ping_endpoints.py` ; fix TTS message format and input script	Real words in, real audio out
S5	Connect the LLM (`LLM_BASE_URL`); measure each stage	Median turn gap at or under 3 s (goal 2.5 s)

Step	Work	Gate
S6	Call control: transfer by SIP REFER, hangup, SMS	Transfer and hangup work from a bot decision
S7	Tune VAD and barge-in on real 8 kHz phone audio	No talk-over; no cut-offs
S8	Load test with N simultaneous scripted callers	Measured capacity and first bottleneck

4.3 Call start, call control and failures

- **Call start:** caller number from the SIP participant and caller details from the external APIs (ported from Rust) seed the session, so the bot does not ask for them again. Session state is keyed by `call_id`.
- **Call control:** `transfer_human` (SIP REFER to the agent desk number or queue), `end_call`, `send_sms` (needs an SMS API; to confirm).
- **Failures:** if the STT, TTS or LLM endpoint fails or is slow, the agent plays a fixed "system problem" message and offers a callback or a human. Worker failure and deploy updates: T5 and T7. This logic lives in the agent.
- **Silence:** a timeout reprompts the caller, then ends the call politely.

4.4 Engine: our own workflow engine

- Workflows are state machines defined in `workflow.yaml` plus `hooks.py`. The LLM only extracts names and helps routing; code owns the flow and every side effect.
- Conversation state is in Postgres (section 5). RAG is plain code: chunking, embedding calls and `pgvector` queries.
- We maintain persistence and resume, the workflow stack, digression handling, slot-filling helpers and verification, with a documented workflow format and scenario tests on every change.
- LiveKit's tasks and handoffs are not used for workflows; its prebuilt DTMF and warm-transfer tasks may be used for call control (T4).
- Revisit if workflows become LLM-planned or maintenance grows with more workflows.

5. Tech stack (proposed)

Layer	Technology	Used to build
Voice framework	LiveKit Agents (Python) with Silero VAD (plugins)	Agent worker, turn-taking, barge-in, STT and TTS plugins
Media server	Self-hosted LiveKit server (repo, self-hosting)	One room per call
SIP service	Self-hosted LiveKit SIP service with Redis (repo, setup)	Taking calls from Asterisk, caller number, transfer by SIP REFER
Telephony	Asterisk PBX trunking SIP into LiveKit (to confirm; trunk setup, inbound calls)	Getting phone calls into LiveKit
Service	Python 3.11, FastAPI	Call session handling, caller-details client, text test endpoint, tooling
STT	Hosted Urdu endpoint, HTTP `chughtai`, one request per utterance	Caller speech to Roman Urdu text
TTS	Hosted Urdu endpoint, websocket	Reply text to speech, with script conversion if needed
LLM	Qwen3-8B on vLLM, thinking off, JSON-schema output	Name extraction and routing help only

Layer	Technology	Used to build
Router	Rules first, then Qwen3-8B	Per-turn decision: continue, faq, lookup, switch, resume, unclear, human, end
Structured knowledge base	Postgres with `pg_trgm` (CSV at the start)	Centers, doctors, tests, home care, center services, cards, promotions; `resolve` and `facts`
Vector store	`pgvector` in the same Postgres, Postgres keyword search, a multilingual embedding model (for example bge-m3)	RAG over approved documents
Data structuring	Offline LLM-assisted extraction, human review, `check_data`	Turning the raw data into approved tables (phase 1)
Logs and eval	Per-call JSONL, scenario runner, knowledge eval set	Testing and replaying calls
Deployment	Docker Compose in `deploy/`	Running LiveKit, SIP, Redis, Postgres and the agent
Workflow orchestration	Our engine: `workflow.yaml` plus `hooks.py`	Slots, states, transitions, confirmations, suspend, resume
LiveKit link	Engine called directly from the agent	Text in; replies and control events out
Conversation state	Postgres session table keyed by call id, with an expiry time (optional in-memory cache per worker)	Workflow stack, slots, history, shared facts
RAG	Plain code	Chunking, embedding calls, `pgvector` queries, thresholds

The dialogue rules (workflow stack, digressions, slot filling, normalizers, verification) are our code. Redis runs only because LiveKit's self-hosted SIP service needs it; conversation state is in Postgres.

6. Engine modules

#	Module	What it does	Prototype	Work to do
E1	Call session	Call start and end, caller details seeding, text-only `/turn` endpoint for tests	`server.py` session and turn endpoints	Add `call_id`, `caller_details`; drop UI-only endpoints
E2	Session store	Workflow stack, slots, history, shared facts	In-memory	Postgres session table with expiry, so a restart does not drop calls
E3	Normalizer and corrector	Phone digits, numbers, dates, names, ASR fixes	`nlp/`	Tune on real STT output
E4	Router	continue, faq, lookup, switch, resume, unclear, human, end	`router/`	Held-out real-utterance set; re-measure
E5	Workflow engine	Runs `workflow.yaml` plus `hooks.py`; suspend, resume, non-interruptible states	`flow/`	Document the format
E6	Workflows	The five categories and sub-workflows (0.3)	Home sampling, booking status	Build doctors, test workflows, home care, center services; real rules

#	Module	What it does	Prototype	Work to do
E7	Knowledge base	Centers, doctors, tests, home care, center services, cards, promotions; aliases; `resolve`, `facts`; per-workflow data access	`datakit/` (tests, areas, branches, pricing)	New entities and relations (7.2); real data; manifests list readable entities
E8	FAQ and promotions	Word-for-word answers; prices and discounts from data	`faq.py`	Real FAQ list; promotions as data
E9	Business API adapters	Booking, status, cancel, appointments	`backend/mock_api.py`	Adapter interface for real APIs; keep mock for tests
E10	LLM client	Qwen3-8B, JSON output	`llm/client.py`	`LLM_BASE_URL`; tune prompts
E11	Reply layer	Templated Roman Urdu; script conversion; pronunciation lexicon	Templates	Confirm TTS input script; if Urdu script, build conversion and test pronunciation
E12	Safety and handoff	Retry limits, confirmation, human transfer	Flow and router	Real transfer target; transfer, hangup, SMS via T4
E13	Logging	Per-call JSONL, PII redacted	`bot._log`	Add call id, timings, router decision
E14	Eval harness	Scenarios, noise eval, router eval, data checks	`scripts/`, `eval/`	Scenarios per new category and from real STT
E15	Data structuring pipeline	Raw data to approved tables with aliases, relations, source reference, validity dates (7.1)	Not built; `check_data.py` validates prototype lists	Extraction, review and load tooling; extend `check_data` for relations and expired promotions
E16	Retrieval (RAG)	Ingest approved documents, hybrid retrieval, grounded answers, refusal on low confidence (7.4)	Not built	Ingestion, `pgvector` index, retrieval with entity and validity filters, knowledge eval set
E17	Workflow registry and scaffolding	Scans manifests, validates and registers workflows; builds the router shortlist; `new_workflow`, `check_workflow` (14)	Manifests and two scripts for two workflows	Extend manifest schema; cross-workflow routing and switching checks; confusable-phrase detection

Voice service modules: V1 LiveKit agent worker and deploy files (server, SIP service, Redis): built, run against fake endpoints; production and SIP not exercised. V2 VAD and barge-in settings: run on simulated audio only. V3 STT plugin (HTTP `chughtai`): `services/stt.py`; real endpoint untested. V4 TTS plugin (websocket) and script conversion: `services/tts.py`; message format is a guess.

Telephony modules:

#	Module	Status
T1	Asterisk to LiveKit SIP trunk, inbound trunk, dispatch rule (room per call)	New; check C2
T2	Caller number from the SIP participant	New; verify the attribute
T3	Caller-details client: external API calls ported from Rust, run at call start	New port; needs API list and auth from the Rust team
T4	Call control: transfer by SIP REFER, hangup, SMS	New port; transfer target and SMS API to confirm
T5	Fallback and error handling in the agent (endpoint failure, silence, worker failure)	New
T6	Recording and consent line	To design, per the compliance decision

#	Module	Status
T7	Health checks, monitoring, worker draining on deploy	New

7. Knowledge base

7.1 Data structuring pipeline (first step)

1. **Collect** raw sources (documents, sheets, web pages, price lists) and record where each came from.
2. **Extract** records into the entity tables. LLM assistance is allowed offline (not during calls); a person reviews every record against its source.
3. **Normalize** names, cities, units, currency. Add Roman Urdu and Urdu aliases, including the spellings the recognizer produces.
4. **Link** test to doctors, doctor to center, discounts and promotions to tests, centers and cards.
5. **Validate** with `check_data`: duplicates, missing aliases or required fields, bad ids, expired promotions, broken links. Bad data blocks the load.
6. **Approve and load.** Each record carries a source reference, last-updated date and approval status; only approved records are spoken.
7. **Refresh** on a defined schedule, keeping the previous version for rollback.

Workflows that depend on an entity cannot be tested on real data until its structuring is done.

7.2 Entities and relations

Entity	Fields	Relations
Centers	name, location, city, address, contact, cards, promotions, discounts	doctors, tests offered, center services
Doctors	name, qualification, fee, timing, days	center(s) (addition), tests
Tests	name, aliases, protocol, charges, discounts, `home_sampling_available`	linked doctors, discounts
Home care services (addition)	name, aliases, charges, availability, areas	none
Center services (addition)	name, aliases, centers offering it, charges, booking rules	center(s)
Cards, promotions, discounts	name, benefit, value, valid from and to, applies to	tests, centers, cards

7.3 Which workflow reads which data

Category	Reads	Writes through API
Doctors	Doctors, Centers	Consultation booking, reschedule, cancel
Home sampling	Tests, areas, Centers, discounts	Home sampling request
Test workflows	Tests, discounts, Doctors	Reschedule, cancel; results status lookup
Home care	Home care services, areas	Home care request
Center services	Center services, Centers, cards	Appointment or request, where it exists

Each manifest lists the entities its workflow may read, so a new data list reaches a workflow by configuration.

7.4 Retrieval (RAG) over documents

Fields cover fees, timings, days and charges. Protocols, preparation notes, promotion terms, policies and long FAQs are text, so retrieval is part of the build.

Answer order: (1) structured facts via `resolve` and `facts`; (2) approved FAQ; (3) retrieval over approved documents; (4) a human.

- **Ingestion** (same approval as 7.1): documents are split into chunks by section with metadata (entity, center, category, valid from and to, source, language), embedded with a multilingual model, and stored in `pgvector` with a keyword index for hybrid search. Indexes are versioned and refreshed on the data schedule.
- **At call time:** query is the normalized text plus any resolved entity; filter by entity and validity date before ranking; two score thresholds give answer, confirm ("Kya aap ... ke baare mein pooch rahe hain?") or refuse. Measure the embedding model's Roman Urdu retrieval quality; if weak, store Roman Urdu keywords and aliases per chunk.
- **Grounding rule:** a spoken answer comes only from retrieved approved text, by default an extractive snippet in a templated reply. Numbers always come from structured facts. Optional, behind a flag and needing approval: the LLM compresses the snippet into one or two sentences, with a validator that every number, name and unit appears in the source; otherwise fall back to the snippet. This is the only place the LLM would write reply text.
- **Where it plugs in:** when the router returns `faq` or `lookup` with no structured or FAQ match, the turn falls through to retrieval. Workflows can call it (for example fasting guidance for a chosen test). Low confidence gives "mujhe is ke baare mein pakka maloom nahi" and an offer of a human. It is a retrieval module the engine calls.
- **Evaluation:** a knowledge eval set of questions with the expected source passage, including Roman Urdu variants, STT-style noise and questions to refuse. Track retrieval hit rate, wrong-answer rate (near zero) and correct-refusal rate.

7.5 Lookup layer

- `resolve(entity, text)` returns `resolved`, `confirm`, `ambiguous` or `none` with ranked candidates. Ambiguous: name up to three options and what differs; still ambiguous: ask one distinguishing attribute, then ask the caller to spell it; after two failed rounds: human.
- `facts(entity, id, fields)` returns only the requested fields, ready to speak. Every fee, timing, price, address and discount comes from `facts`.
- `check_data` validates duplicates, missing aliases and bad ids on every load. Adding a list means adding a data file and a config file, not code.

8. Workflows and switching

A workflow is a folder: `manifest.yaml` (description, example phrases), `workflow.yaml` (slots, states, transitions, confirmations), `hooks.py` (optional), prompts, tools and scenarios. Adding one does not change the core. Categories and sub-workflows: 0.3; data each reads: 7.3.

- **Information sub-workflows** (doctor details, test rates, protocols, discounts, center and home care information): resolve the item, read the stored fact, return to what was pending. No stack.
- **Home sampling (generic):** one workflow for every case. Slots: tests, patient name, age, gender, address, date and time slot, phone (caller ID first), fasting acknowledgement, notes. Tests that need a lab visit (imaging, ECG, biopsy) are explained, not booked. Read back every high-risk slot, then submit only after explicit confirmation.
- **Booking sub-workflows** (consultation, home care request, center service request): the same pattern: slots, read-back, explicit confirmation, one API call.

- **Reschedule, cancel, results inquiry:** the caller gives the booking reference and the phone number on the booking, matched exactly and read back digit by digit, before details are spoken or anything changes. Results inquiry is the most sensitive; its verification rule needs the lab's approval. The prototype's `booking_status` is the starting point.
- **Overlap to settle:** reschedule and cancel sit under Test workflows but apply to home sampling bookings too: one shared workflow or one per category.

Router decisions each turn: `continue`, `faq`, `lookup`, `switch`, `resume`, `unclear`, `human`, `end`.

Switching rules: bias to `continue` (a misheard word must not abandon a booking); suspend, never discard, and offer to resume afterwards; maximum two suspended workflows, then offer a human; confirm before leaving a half-filled workflow; final read-back and submit cannot be interrupted; shared facts (name, phone) carry across and are not asked twice.

8.1 How switching works on a call

State kept per call (survives a worker restart, section 5): the active workflow and its pending question; a stack of suspended workflows (maximum two) with slots, retry counts and pending question; shared facts (name, phone, caller ID, caller details); a flag for non-interruptible steps.

Every turn: 1. The caller's text is normalized. 2. The router decides in order: **cheap pass** (rules, no model call: is it a plausible answer to the pending question? then `continue`); **switch cues** (explicit phrases such as "pehle ...", "nahi, mujhe ... karna hai", plus manifest keywords and example phrases); **model pass** (Qwen3-8B with structured output over a shortlist of best-matching workflows, returning decision, target and confidence); **policy in code** (low confidence gives one clarifying question; leaving a workflow with data needs spoken confirmation; non-interruptible steps block any switch; two unclear turns offer a human). 3. The decision runs: `continue` (active workflow takes the answer); `faq` / `lookup` (answer, then re-ask the pending question; stack unchanged); `switch` (confirm, suspend onto the stack with slots and pending question, start the target seeded with shared facts); `resume` (restore slots, re-ask the pending question); `human` / `end`. 4. State is saved and the reply is spoken.

When the new workflow ends or is dropped, the bot pops the stack and offers to resume; if the caller declines, the suspended workflow is discarded. A third switch while two are suspended offers a human. A caller can reach any enabled workflow from any state, including at call start, because the router runs before the active workflow every turn.

```
Bot   : Sample kis address par lena hai?
Caller: Dr Ahmed ki fee kitni hai?                  -> lookup; stack unchanged
Bot   : Dr Ahmed ki fee {fee} rupay hai. Sample kis address par lena hai?
Caller: mujhe doctor ka appointment bhi chahiye     -> switch cue; booking has data
Bot   : Aapki home sampling booking adhoori hai. Pehle doctor ka appointment le lein?
Caller: haan                                        -> suspend home_sampling, start
doctor_booking
Bot   : Kaun se doctor se milna hai?
...   (doctor booking confirmed and submitted)
Bot   : Ab home sampling booking jari rakhein? Sample kis address par lena hai?
```

Hard cases - **Misheard words:** one low-confidence turn never switches; a switch needs an explicit phrase, or high confidence plus a spoken confirmation; when unsure, prefer `continue`. - **Overlapping workflows:** proposed default is one shared `booking_changes` workflow that works out the booking type from the reference and asks if it cannot (open, section 16). - **Barge-in:** speech is cut off and the cut-off reply counts as not delivered; the new utterance is the next turn, and the pending question is re-asked unless the utterance answers it. During a non-interruptible step the bot repeats the read-back instead of switching. - **Data across workflows:** name and phone are not asked again; workflow-specific slots are not shared. - **Verification:** reschedule, cancel and results inquiry still need the booking reference and phone, even when reached by a switch. - **Router latency:** rules run first and the model pass runs only for turns they do not decide. The share decided by rules and the model-pass latency are measured on real calls.

Engine implementation: the Postgres session record holds the active workflow, the suspended stack, shared facts and pending question. The engine pushes a workflow's state onto the stack to suspend it and pops to restore it.

Switching tests: scenarios for every workflow pair (A to B and back, B to A, switch declined, false switch from a misheard word, switch during read-back, two switches and a third, shared facts not asked twice, slots intact after resume, suspended workflow abandoned); a router test set from real utterances with noisy variants (targets in section 12); review of wrong or missed switches in the shadow pilot.

9. Handling bad STT

1. Read back every high-risk slot (name, phone, address, tests, date and time).
2. Phone: use caller ID; otherwise collect in chunks and read back digit by digit.
3. Snap tests, doctors, centers and areas to closed lists ("pick the closest option").
4. Two strikes per slot: rephrase once, then offer a human.
5. Empty or very short transcript: ask the caller to repeat.
6. Names are stored as heard and flagged for human verification unless the caller confirmed.
7. Log raw transcript, corrected text, router decision, slots and reply every turn, so failures can be replayed.

10. Safety and compliance rules

- Every fee, timing, price, address and discount is read from data.
- Never submit without explicit caller confirmation.
- Medical advice, result interpretation and complaints go to a human.
- Outside workflows and the knowledge base: say it cannot help and offer a human; never guess.
- Retrieved answers come only from approved documents (7.4).
- The caller can ask for a human at any time.
- Logs redact PII by default; recording consent line, retention period and access control to be defined.
- Early launch: the bot collects the request and a human verifies by callback, until measured accuracy allows removing the callback.

11. Latency budget

Target: caller stops speaking to bot starts speaking, at or under 3 s median (goal 2.5 s). Stages to measure in phase 3: end-of-speech detection, STT per utterance, normalizer and corrector, router and LLM, retrieval (RAG turns), TTS first audio. Per-stage budgets are set from the first measurements on the real endpoints. Use spoken filler lines for slow steps.

12. Evaluation and targets

Metric	Proposed target
Routing accuracy on noisy text	At least 90%
Wrong switches	At most 1%
Switching scenarios	100% pass on clean text, at least 90% on noisy; no switch during non-interruptible states; resumed workflows keep slots
Slot accuracy	Tests at least 90%; phone at least 95% after confirmation
Booking completion without a human	At least 80%
Wrong data submitted; wrong facts in answers	About zero

Metric	Proposed target
Retrieval (RAG)	Right passage on the knowledge eval set (target set from first measurement); wrong answers about zero; unanswerable questions refused
Forced-failure cases reaching a human instead of looping	100%
Median and p95 turn latency	At most 3 s and 4 s (goal 2.5 s median)

Test assets: 30+ scenarios per workflow including switching, noisy variants from real STT output, silence, refusals, abusive callers and endpoint-down cases; a held-out routing set from real calls; per-list entity resolution sets.

13. Build phases

Each phase ends with a review gate. **Phase 1, data structuring, is the first step**: the raw data is unstructured and every spoken fact depends on it. Phases 4 onward need approved tables. Phase 2 does not depend on the data, so a different person can run it alongside phase 1. Otherwise, do not start a phase until the previous one is signed off.

Phase	Work	Gate
1	**Data structuring.** Inventory raw sources and agree table schemas (0.1, 7.2). Then one entity at a time in the order the workflows need it (tests, doctors, centers, then home care, center services, cards, promotions): extract, review against source, normalize, add aliases, link, `check_data`, approve, load. Also collect the real FAQ list and the documents for RAG.	Each entity approved by the data owner and `check_data` passes; first slice (tests, doctors, centers) available early so workflow work can start
2	Stand up LiveKit, SIP and Redis (S1); run C1 and C2 (4.2); agent with a stub engine	C1 complete; C2 passes (call connects, caller number arrives, transfer works); a real call hears a canned reply
3	Connect the real LLM and STT/TTS tokens; `ping_endpoints.py`; fix the TTS message format; measure stage latencies	Real words in, real audio out; latency known
4	Postgres sessions; document the workflow format and manifest schema; build the registry, scaffold and `check_workflow` (14); run the two existing workflows on the real data from phase 1	Scenarios pass; state survives an engine restart
5	Feed real STT output through; rebuild noise scenarios from it; tune corrector and router; build the held-out routing set	Section 12 targets met on real text
6	Build the remaining workflow categories one at a time (doctors, test workflows, home care, center services) and the business API adapters; re-run switching across all workflows after each	Each category's scenarios pass; switching scenarios pass
7	Fact questions from structured data, then RAG over approved documents (7.4): ingestion, retrieval, grounding rules, knowledge eval set	No wrong facts on the test set; retrieval and refusal targets met
8	Safety, call control (transfer, end, SMS), fallback, TTS wording by listening	Failure cases reach a human
9	Load test with N simultaneous scripted callers; size endpoints and workers	Capacity and first bottleneck known
10	Shadow pilot on real calls; review failures, including wrong or missed switches; add aliases and scenarios	Pilot results reviewed
Contingency	If C2 fails, keep the existing Rust layer as the audio gateway with a Pipecat voice service over a WebSocket link (4.2)	Phone call completes a mock booking

14. Adding a workflow (plug-in design)

Goal: adding a workflow means adding a folder; the router, session manager and voice code do not change. A form-like workflow (collect slots, confirm, call one API) is defined in YAML; custom validation and an API adapter are small code.

Registry. At startup the system scans `workflows/*/manifest.yaml`, validates each, and registers it. A manifest that fails validation is not registered and the failure is logged. A workflow is live only when `enabled: true`.

Proposed manifest (fields to confirm):

```
name: home_care
version: 1
enabled: false                    # true only after check_workflow passes
title: Home care services
description: Nurse visit, injections and drips at the patient's home
category: home_care
example_phrases:                  # 10 to 20 Roman Urdu phrases incl. STT-style spellings
  - "nurse ghar bhejna hai"
  - "injection ghar par lagwana hai"
keywords: [nurse, injection, drip, ghar]
reads: [home_care_services, areas]   # knowledge entities it may read
tools: [create_home_care_request]    # API calls it may make
requires_verification: false         # true: booking reference + phone first
interruptible: true                  # per-step overrides in the definition
shared_facts_used: [name, phone, address]
max_retries_per_slot: 2
```

The router reads the registry. Its shortlist and model prompt are built from the manifests (descriptions, example phrases, keywords), so a new manifest is all the router needs. The shortlist is fixed at the best three to five matches. Reaching a workflow from any point follows 8.1.

Steps to add a workflow 1. Write the goal, slots, final action, and whether it needs identity checks. 2. Scaffold: `new_workflow.py <name>` creates the folder with manifest, workflow definition, prompts, tools and scenario templates. 3. Fill in the manifest with 10 to 20 example phrases and keywords. 4. Add or reuse knowledge lists and aliases; run `check_data`. 5. Connect the API tool (mock first). 6. Write 30+ scenarios, including switching to and from every existing workflow. 7. Run `check_workflow <name>`: it validates manifest, definition, knowledge references and scenarios, then runs routing and switching tests across all workflows and reports against section 12. 8. Set `enabled: true` deliberately, after trying it in shadow mode on real calls.

Guardrails as workflows grow - Routing accuracy and false-switch rate must still meet section 12 for all workflows after each addition; if they drop, add example phrases, merge overlapping workflows, or train the router. - `check_workflow` flags manifests whose example phrases are close to another workflow's. - A workflow cannot read entities or call tools it has not declared. - A change raises the workflow's version and re-runs its checks. A failing workflow cannot be enabled.

Design is done when a new form-like workflow goes from description to passing `check_workflow` with no change to the router, session manager or voice code. The target is under one working day for a config-only workflow. The prototype has manifests and the `new_workflow.py` and `check_workflow.py` scripts; they need extending for the fields above and the cross-workflow checks.

15. Risks

Risk	Impact	Mitigation
STT errors on real calls	Wrong name, address, test	Closed-list snapping, read-back, human verification callback, corrector
Real endpoints never tested	Latency and accuracy unknown	Phase 3 early

Risk	Impact	Mitigation
Router weak on real utterances	Misroutes, false switches	Held-out real set, bias to continue, confirm before switching
Qwen3-8B drift	Wrong extraction	Narrow use (names, routing), JSON schema, scenario evals on every change
Sample data in the prototype	Wrong facts if shipped	Replace before any pilot; `check_data` warns on sample content
Unstructured source data; extraction errors; data changes after load	Wrong, missing or stale facts; workflows blocked until data exists	Structure first (7.1); human review against source; source reference and validity dates; scheduled refresh
Slow or down endpoints	Silence, dropped calls	Fallback message, callback offer, timeouts
Patient data in logs	Compliance	Redaction, retention policy, access control
Asterisk to LiveKit SIP path unverified; SIP status unclear	Calls fail or poor audio	C2 first; contingency to keep Rust; test SIP connectivity before debugging the agent
Rust functions not inventoried	Needed features missing from the Python build	C1 with the Rust team; shadow run before go-live
LiveKit infrastructure (server, SIP, Redis, public IP, ports)	Operations load; NAT or firewall failures	Compose files in `deploy/`; health checks; dev host with a public IP early
Postgres read and write per turn	Added latency (unmeasured)	Measure in phases 3 and 4; in-memory cache per worker if needed
RAG returns a wrong passage; embedding model weak on Roman Urdu	Wrong spoken guidance; poor retrieval	Approved sources only, entity and validity filters, thresholds, extractive answers, number validator, knowledge eval set, human fallback; measure early, add Roman Urdu keywords per chunk
We maintain the whole engine	Burden as workflows grow	Document the format; scenario tests on every change; revisit the engine choice

16. Open decisions

- **Telephony:** confirm Asterisk stays and trunks into LiveKit SIP; the pilot host with a public IP and open ports (UDP 5060 and 10000 to 20000); where human transfers and callbacks go.
- **Rust:** which external APIs it calls for caller details (endpoints, auth, response format); everything else it does (C1).
- **STT and TTS:** STT output is Roman Urdu (confirmed); are partial results available or only final transcripts? Does the TTS take Roman Urdu or Urdu script (Urdu script needs conversion, E11)?
- **Data:** what the raw data looks like (formats, volume, where it lives) and who approves the structured version; confirm the 0.1 additions; refresh rate of center, doctor, test and promotion data.
- **Workflows:** sub-workflows for categories 1, 2, 4 and 5; where reschedule and cancel live; the verification rule for results inquiry.
- **Backend:** whether real booking and appointment APIs exist, and who owns them.
- **RAG:** the embedding model and its Roman Urdu quality; whether the LLM may compress retrieved text (7.4) or answers stay extractive.
- **Compliance and scale:** recording consent, PII retention; expected call volume and peak simultaneous calls (sets phase 9).
- **Engine language:** Python (decided).

17. LiveKit references and examples

Found by web search while preparing this plan; not opened or validated one by one. Check each against the version you pin.

Topic	Link
Telephony overview	https://docs.livekit.io/telephony/
Accepting inbound calls	https://docs.livekit.io/telephony/accepting-calls/
Inbound call workflow setup	https://docs.livekit.io/telephony/accepting-calls/workflow-setup/
Dispatch rules (room per caller, agent dispatch)	https://docs.livekit.io/telephony/accepting-calls/dispatch-rule/
SIP trunk setup	https://docs.livekit.io/telephony/start/sip-trunk-setup/
Cold transfer by SIP REFER	https://docs.livekit.io/telephony/features/transfers/cold
Telephony with agents	https://docs.livekit.io/frontends/telephony/agents/
Telephony commands in the CLI	https://docs.livekit.io/reference/developer-tools/livekit-cli/telephony/
Self-hosting overview, SIP service (Redis, public IP, ports), deployment	https://docs.livekit.io/transport/self-hosting/, https://docs.livekit.io/transport/self-hosting/sip-server/, https://docs.livekit.io/transport/self-hosting/deployment.md
Agents plugins (custom STT and TTS plugins)	https://docs.livekit.io/agents/plugins
Source: LiveKit server, SIP service	https://github.com/livekit/livekit, https://github.com/livekit/sip
Example: call answering agent (related to S2)	https://docs.livekit.io/reference/recipes/answer_call
Example: outbound calls (callbacks)	https://docs.livekit.io/recipes/make_call/
Examples: Python voice and telephony agents	https://github.com/livekit-examples/python-agents-examples, https://github.com/livekit-examples/python-agents-examples/tree/main/telephony
Example: phone assistant	https://github.com/livekit-examples/phone-assistant
Third-party article: LiveKit with Asterisk	https://medium.com/@nksaqlain/livekit-asterisk-pbx-cartesia-openai-deepgram-creating-your-first-ai-telephony-agent-3dbafe55f185
LiveKit's comparison with Pipecat (vendor source)	https://livekit.io/field-guides/guide/livekit-vs-pipecat
Pipecat transports (contingency, 4.2)	https://docs.pipecat.ai/guides/learn/transports

Prototype in this repo: `voicebot/app/agent.py`, `voicebot/app/services/` (STT and TTS plugins), `voicebot/deploy/` (compose and configs), `voicebot/scripts/e2e_voice_fake.py` (end-to-end test with a simulated caller).